Microservices – Detailed Notes

Scope: From basics (Monolith vs Microservices) to advanced patterns (Eureka, Feign, Config Server, API Gateway, Circuit Breakers with Resilience4j).

---

1) Introduction to Microservices

1.1 Monolith vs Microservices

- **Monolithic architecture**: single deployable unit, tightly coupled modules, single database.
- **Microservices architecture**: system decomposed into small, independently deployable services, each with its own business logic and often its own database.

1.2 Advantages of Microservices

- Independent development and deployment.
- Technology diversity (polyglot programming).
- Fault isolation.
- Scalability at service level.

1.3 Challenges of Microservices

- Distributed system complexity.
- Network latency and failures.
- Data consistency and transactions.
- Centralized logging, monitoring, and debugging.
- DevOps maturity required (CI/CD, containers, orchestration).

---

2) Service Discovery

2.1 What is Service Discovery?

- Mechanism by which microservices locate each other in a dynamic environment.
- Required because services' IPs/ports change in containerized/auto-scaling environments.

2.2 Eureka Server

- Netflix OSS service registry.
- Provides self-registration & heartbeat mechanism.

Setup:

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-netflix-eureka-server</artifactId>
</dependency>
```

```
@EnableEurekaServer
@SpringBootApplication
public class EurekaServerApp {
  public static void main(String[] args) {
SpringApplication.run(EurekaServerApp.class, args); }
}
```

2.3 Registering Services in Eureka

```
spring:
  application:
    name: order-service
  cloud:
    discovery:
      enabled: true

server:
  port: 8081

eureka:
  client:
    service-url:
      defaultZone: http://localhost:8761/eureka/
```

---

3) Inter-Service Communication

3.1 Synchronous vs Asynchronous

- **Synchronous**: REST, gRPC (request/response, tight coupling).
- **Asynchronous**: messaging systems (Kafka, RabbitMQ) – decouples services.

3.2 Using RestTemplate (deprecated, but common)

```
@Bean
RestTemplate restTemplate(RestTemplateBuilder builder) { return
builder.build(); }
```

```
@Autowired RestTemplate restTemplate;
Order order = restTemplate.getForObject("http://payment-service/payments/123",
Order.class);
```

3.3 Using Feign Client (recommended)

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-openfeign</artifactId>
</dependency>
```

```
@FeignClient(name = "payment-service")
public interface PaymentClient {
  @GetMapping("/payments/{id}")
  Payment getPayment(@PathVariable("id") String id);
}
```

- Eureka integrates with Feign to resolve `payment-service` hostname dynamically.

---

4) Config Server

4.1 What is a Config Server?

- Centralized configuration management.
- Externalizes service configuration for different environments.

4.2 Creating In-Memory Config Server

```
@EnableConfigServer
@SpringBootApplication
public class ConfigServerApp { }
```

```
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/your-org/config-repo
```

4.3 Config Server with GitHub

- Store `application.yml`, `order-service.yml`, etc. in Git repo.
- Clients fetch configs from `http://config-server:8888/order-service/dev`.

---

5) API Gateway

5.1 What is an API Gateway?

- Entry point to microservices system.
- Handles cross-cutting concerns: routing, load balancing, authentication, rate limiting.

5.2 Spring Cloud Gateway Setup

```
<dependency>
  <groupId>org.springframework.cloud</groupId>
  <artifactId>spring-cloud-starter-gateway</artifactId>
</dependency>
```

```
spring:
  cloud:
    gateway:
      routes:
        - id: order-service
          uri: lb://order-service
          predicates:
            - Path=/orders/**
```

5.3 Routing Requests to Services

- `lb://` prefix enables load-balancing via Eureka.
- Gateway forwards requests to registered service instance.

---

6) Circuit Breaker

6.1 Why Circuit Breaker?

- Protects services from cascading failures.
- Monitors calls: if too many failures, trips the circuit (open state).
- After cool-off, allows some test calls (half-open).
- Prevents resource exhaustion.

6.2 Resilience4j Setup

```
<dependency>
  <groupId>io.github.resilience4j</groupId>
  <artifactId>resilience4j-spring-boot3</artifactId>
</dependency>
```

6.3 Using CircuitBreaker Annotation

```
@CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackPayment")
public PaymentResponse getPayment(String id) {
   return paymentClient.getPayment(id);
}

public PaymentResponse fallbackPayment(String id, Throwable ex) {
   return new PaymentResponse("Fallback response due to: " + ex.getMessage());
}
```

6.4 Thresholds & Managing Circuit

```
resilience4j:
  circuitbreaker:
    instances:
      paymentService:
        slidingWindowSize: 10
        failureRateThreshold: 50
        waitDurationInOpenState: 10s
```

---

7) Real-time Considerations

- **Distributed Tracing**: Zipkin/Sleuth for request flow visualization.
- **Logging**: Centralized with ELK/EFK stack.
- **Monitoring**: Prometheus + Grafana.
- **Security**: Secure each service with JWT and Gateway filters.
- **Testing**: Contract testing (e.g., Pact) to ensure inter-service compatibility.

---

8) Quick Recap (Cheatsheet)

- **Eureka**: service registry.
- **Feign**: declarative REST client.
- **Config Server**: centralized config.

- **Gateway**: single entry point.
- **Resilience4j**: circuit breaker & resilience patterns.

Together, these form the backbone of a production-ready microservices system.